FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

# LABEXER3

| | |
|---|---|
| Kurt Lester Carolino | Doc. Hadji Tejuco |
| Name of Student | Name of Professor |
| November 3, 2026 | November 3, 2026 |
| Data Performed | Date Submitted |

MIDTERM EXAM 1

FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

**List the steps that you perform to come up with the program.**

Analyzed the COR layout and Menu requirements: Divided the registration form into three parts—student info at the top (user input), subjects and schedules in the middle (fixed text), and assessment fees at the bottom (user input and math calculations)—and designed a 5-option interactive menu (1. Input, 2. Output, 3. Write, 4. Read, 5. Exit) to control the program flow.

Set up the .data and .bss sections: Placed all text prompts, menu strings, file status messages, labels, the fixed schedule table, the default output file descriptor (out_fd set to 1), and the constant tuition fee (44,268.00) inside .data. Reserved empty memory buffers for the menu choice, user-specified filename, student details, math results, and a 4096-byte file-reading buffer inside .bss.

Created Macros for repeated tasks: Wrote macros (PRINT_ZSTR, PROMPT_TEXT, PROMPT_FEE, and PRINT_FEE_ROW) to handle printing text and reading keyboard input so Linux system calls (int 0x80) did not have to be rewritten repeatedly.

Handled decimal math using centavos: Multiplied all money values by 100 (converting pesos to centavos) so 32-bit assembly integer registers could accurately calculate .00, 40%, and 30% decimals without floating-point errors.

Implemented the Menu Loop and Modular Procedures: Used a continuous loop (.menu_loop) in _start and call/ret procedures to organize each option: getting inputs (get_user_inputs), computing totals and installments (calculate_assessment), displaying the full COR (display_full_info), writing the COR to a text file (write_to_file), and reading any specified file in the folder (read_from_file).

Implemented Linux File Handling (sys_creat, sys_open, sys_read, sys_write, sys_close): Updated print_zstr to write to [out_fd] so the program can switch between printing to the terminal screen (out_fd = 1) and writing directly to cor_output.txt (Option 3), and added a file reader that asks the user for a filename in the directory to open and display on the screen (Option 4).

Tested and formatted the output: Assembled and linked the program in the Linux terminal using nasm and ld, then verified that the menu options, two-column student header, fee table, and file read/write operations worked properly.

**Paste the source code and explain the functionalities and take a screenshot of the final output.**

.data and .bss Sections: The .data section stores fixed strings such as the COR menu prompt, file operation messages, student info questions, the ASCII schedule table, the active output file descriptor (out_fd), and the fixed tuition fee (4426800 centavos). The .bss section reserves uninitialized memory buffers for the user's menu selection (menu_choice), the user-specified filename (read_filename), the file descriptor (in_fd), the 4096-byte file-reading buffer (file_buf), the student's typed text, and variables for the calculated fees.

**Macros (%macro):**

PRINT_ZSTR: Passes a string address in esi to the print_zstr procedure to output text to the active file descriptor (out_fd).

PROMPT_TEXT: Prints a prompt, reads what the user types from the keyboard using sys_read (eax = 3, ebx = 0), and calls strip_newline to remove the trailing Enter key.

PROMPT_FEE: Reads a fee amount from the user, converts the text into centavos via parse_currency, and saves the integer in a memory variable.

PRINT_FEE_ROW: Prints a fee label alongside its formatted currency value and a newline.

**Main Menu & Procedures (call/ret):**

_start (.menu_loop): Displays the interactive COR menu (1. Input, 2. Output, 3. Write, 4. Read, 5. Exit), reads the user's choice, and jumps to the selected operation before looping back until 5 (sys_exit, eax = 1) is selected.

get_user_inputs (Option 1 - Input): Asks the user to type their personal details (Status, Student #, Name, Phone, Zip Code, Classification, College, Program, Year, Term & SY, Address) and variable assessment fees (Miscellaneous, ID Validation, and Laboratory fees).

calculate_assessment (Option 1 - Input): Adds the fixed Tuition Fee (44,268.00) and all user-entered fees to get total_assessment, then uses mul and div to compute the Down Payment (Total×40÷100), Midterm (Total×30÷100), and Final payment (Total×30÷100).

display_full_info, print_top_section, print_schedule_section, and print_assessment_section (Option 2 - Output): Calls all three printing sections in sequence—aligning the student details into left and right columns, displaying the static schedule table, and printing the final assessment breakdown.

write_to_file (Option 3 - Write): Creates or overwrites cor_output.txt using sys_creat (eax = 8, permissions 0644o), redirects [out_fd] to the file descriptor, calls display_full_info to write the entire COR into the file, closes the file with sys_close (eax = 6), and resets [out_fd] back to terminal output (1).

read_from_file (Option 4 - Read): Prompts the user to enter a filename located in the directory (read_filename), opens that file in read-only mode using sys_open (eax = 5, ecx = 0), reads its contents in chunks into file_buf using sys_read (eax = 3), prints the contents to the screen using sys_write (eax = 4, ebx = 1), and closes the file with sys_close (eax = 6).

**Helper Procedures:**

print_zstr: Calculates the string length and writes the string to [out_fd] (either the terminal screen or an open file) using sys_write (eax = 4).

parse_currency: Converts a typed fee string (like "12,306.00") into an integer in centavos (1230600) for arithmetic operations.

print_currency: Converts calculated centavo integers back into readable currency format by inserting the decimal point (.) and commas (,) before printing.

pad_to_col_56: Counts how many characters were printed on the left column of the student info header and prints spaces so the right column lines up at column 56.

**Program Description**

This program is a menu-driven 32-bit x86 NASM Assembly application that generates, displays, saves, and reads a student's Certificate of Registration (COR) and assessment of fees. Using macros, modular procedures, and Linux system calls (int 0x80), the program provides five options: entering student details and assessment fees (Input), displaying the formatted COR and calculated 40%/30%/30% payment schedule on the screen (Output), saving the full COR details into a text file (Write), prompting for a filename in the directory to read and display its contents (Read), and terminating the program cleanly (Exit).

**PROGRAM OUTPUT:**

```
====================================
                COR
====================================
1. Input
2. Output
3. Write
4. Read
5. Exit
Enter your choice: 2

Enrollment Status : ENROLLED
Student # : 202510161                          College : COMPUTER STUDIES   Program : BSCSSE
Name : CAROLINO, KURT LESTER AGCAOILI          Year Level : 1   1st term, SY 26-27
Phone : +639913575840    Zip Code : 1485       Address : 335 TAGANAHAN STREET, BAGUMBAYAN SOUTH
Classification : N/A

+-----------------------------------------------------------------------------------------------------------------------------------+
| Courses   Title                                Section   Units   Days    Time                                    Room           |
+-----------------------------------------------------------------------------------------------------------------------------------+
| CCS0021   INFORMATION MANAGEMENT (LEC)         TN24        2     T       16:00:00-18:40:00                       ONLINE         |
| CCS0021L  INFORMATION MANAGEMENT (LAB)         TN24        1     F       16:00:00-18:50:00                       ONLINE         |
| CS0001    DISCRETE STRUCTURES 1                TN24        3     S / W   15:00:00-16:50:00 / 15:00:00-16:50:00   F710 / F710    |
| CS0003    COMPUTER SYSTEMS & ARCHITECTURE      TN24        2     F       10:00:00-12:40:00                       ONLINE         |
| CS0003L   COMPUTER SYSTEMS & ARCHITECTURE (LAB) TN24       1     TH      10:00:00-12:50:00                       E610           |
| CS0070    OBJECT ORIENTED PROGRAMMING          TN24        2     F       07:00:00-09:40:00                       ONLINE         |
| CS0070L   OBJECT ORIENTED PROGRAMMING LAB      TN24        1     TH      07:00:00-09:50:00                       E609           |
| GED0021   SPECIALIZED ENGLISH PROGRAM 2        TN24        3     S / W   11:00:00-12:50:00 / 11:00:00-12:50:00   F607 / F607    |
| GED0085   MATHEMATICAL ANALYSIS                MS03        3     S / W   09:00:00-10:50:00 / 09:00:00-10:50:00   F712 / F712    |
| GED0081   COLLEGE PHYSICS 1 LECTURE            TN22        2     F       13:00:00-15:40:00                       ONLINE         |
| GED0081L  COLLEGE PHYSICS 1 LABORATORY         TN22        1     TH      13:00:00-15:50:00                       F1003          |
|                                                                                                                                   |
| TOTAL UNITS 21                                                                                                                    |
+-----------------------------------------------------------------------------------------------------------------------------------+
```

**Write output:**

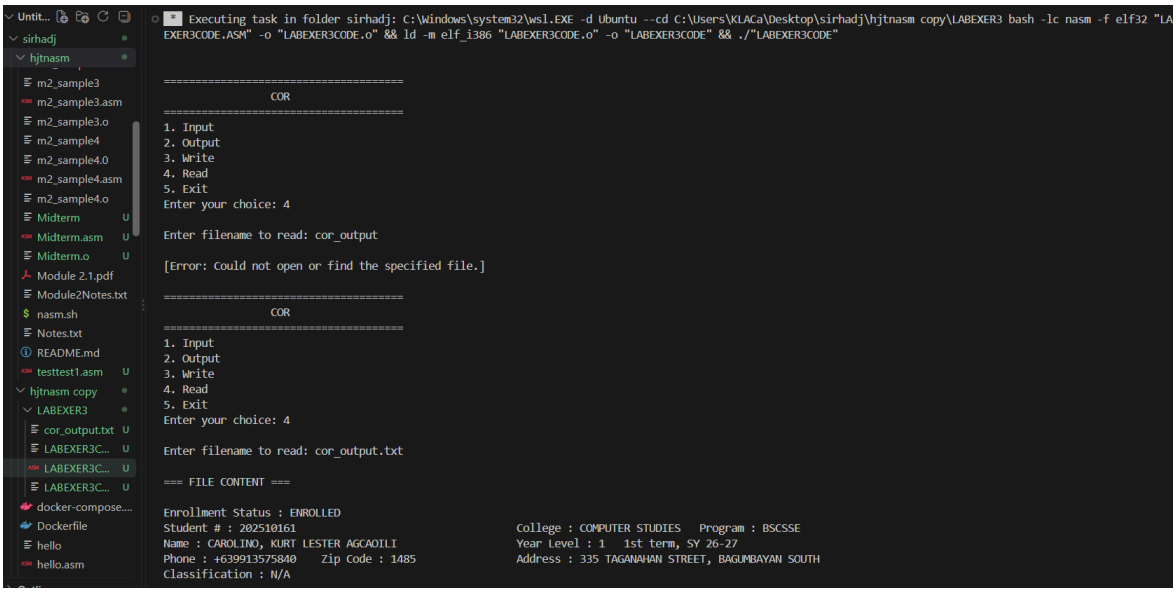
hjtnasm copy > LABEXER3 > cor_output.txt

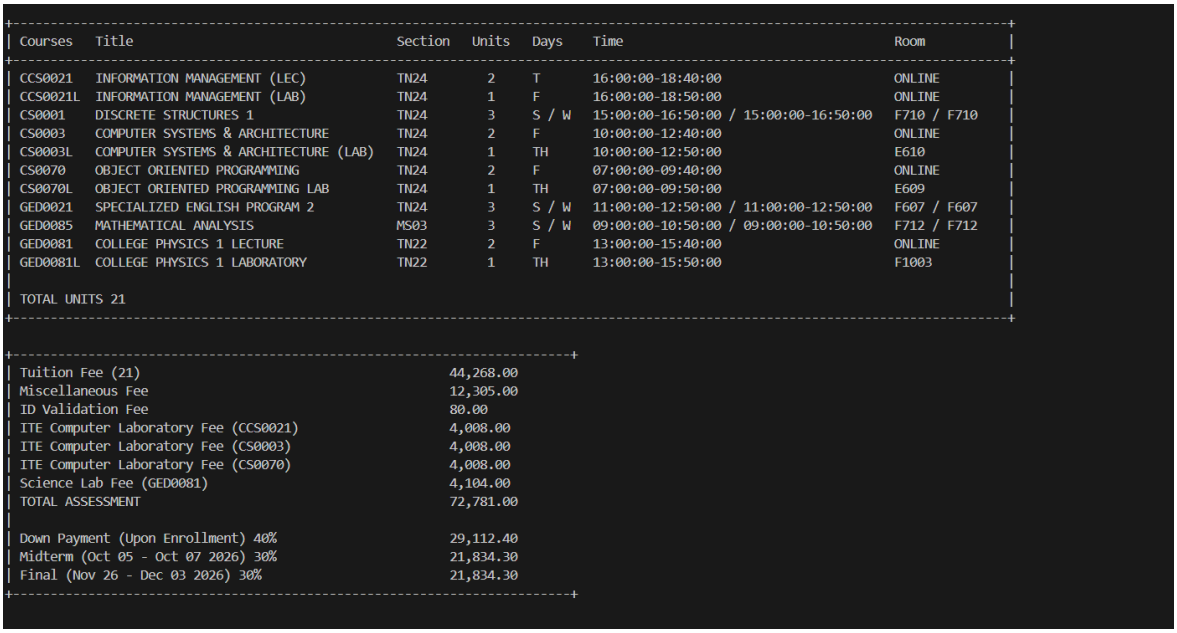
```
Enrollment Status : ENROLLED
Student # : 202510161                          College : COMPUTER STUDIES   Program : BSCSSE
Name : CAROLINO, KURT LESTER AGCAOILI          Year Level : 1   1st term, SY 26-27
Phone : +639913575840    Zip Code : 1485       Address : 335 TAGANAHAN STREET, BAGUMBAYAN SOUTH
Classification : N/A

+-------------------------------------------------------------------------------------------------------------------------------+
| Courses   Title                                Section   Units   Days    Time                                  Room           |
+-------------------------------------------------------------------------------------------------------------------------------+
| CCS0021   INFORMATION MANAGEMENT (LEC)         TN24        2     T       16:00:00-18:40:00                     ONLINE         |
| CCS0021L  INFORMATION MANAGEMENT (LAB)         TN24        1     F       16:00:00-18:50:00                     ONLINE         |
| CS0001    DISCRETE STRUCTURES 1                TN24        3     S / W   15:00:00-16:50:00 / 15:00:00-16:50:00 F710 / F710    |
| CS0003    COMPUTER SYSTEMS & ARCHITECTURE      TN24        2     F       10:00:00-12:40:00                     ONLINE         |
| CS0003L   COMPUTER SYSTEMS & ARCHITECTURE (LAB) TN24       1     TH      10:00:00-12:50:00                     E610           |
| CS0070    OBJECT ORIENTED PROGRAMMING          TN24        2     F       07:00:00-09:40:00                     ONLINE         |
| CS0070L   OBJECT ORIENTED PROGRAMMING LAB      TN24        1     TH      07:00:00-09:50:00                     E609           |
| GED0021   SPECIALIZED ENGLISH PROGRAM 2        TN24        3     S / W   11:00:00-12:50:00 / 11:00:00-12:50:00 F607 / F607    |
| GED0085   MATHEMATICAL ANALYSIS                MS03        3     S / W   09:00:00-10:50:00 / 09:00:00-10:50:00 F712 / F712    |
| GED0081   COLLEGE PHYSICS 1 LECTURE            TN22        2     F       13:00:00-15:40:00                     ONLINE         |
| GED0081L  COLLEGE PHYSICS 1 LABORATORY         TN22        1     TH      13:00:00-15:50:00                     F1003          |
|                                                                                                                               |
| TOTAL UNITS 21                                                                                                                |
+-------------------------------------------------------------------------------+

+-------------------------------------------------------------------+
| Tuition Fee (21)                                    44,268.00
| Miscellaneous Fee                                   12,305.00
| ID Validation Fee                                   80.00
| ITE Computer Laboratory Fee (CCS0021)               4,008.00
| ITE Computer Laboratory Fee (CS0003)                4,008.00
| ITE Computer Laboratory Fee (CS0070)                4,008.00
| Science Lab Fee (GED0081)                           4,104.00
| TOTAL ASSESSMENT                                    72,781.00
|
| Down Payment (Upon Enrollment) 40%                  29,112.40
```

# Read:





| Courses | Title | Section | Units | Days | Time | Room |
|---|---|---|---|---|---|---|
| CCS0021 | INFORMATION MANAGEMENT (LEC) | TN24 | 2 | T | 16:00:00-18:40:00 | ONLINE |
| CCS0021L | INFORMATION MANAGEMENT (LAB) | TN24 | 1 | F | 16:00:00-18:50:00 | ONLINE |
| CS0001 | DISCRETE STRUCTURES 1 | TN24 | 3 | S / W | 15:00:00-16:50:00 / 15:00:00-16:50:00 | F710 / F710 |
| CS0003 | COMPUTER SYSTEMS & ARCHITECTURE | TN24 | 2 | F | 10:00:00-12:40:00 | ONLINE |
| CS0003L | COMPUTER SYSTEMS & ARCHITECTURE (LAB) | TN24 | 1 | TH | 10:00:00-12:50:00 | E610 |
| CS0070 | OBJECT ORIENTED PROGRAMMING | TN24 | 2 | F | 07:00:00-09:40:00 | ONLINE |
| CS0070L | OBJECT ORIENTED PROGRAMMING LAB | TN24 | 1 | TH | 07:00:00-09:50:00 | E609 |
| GED0021 | SPECIALIZED ENGLISH PROGRAM 2 | TN24 | 3 | S / W | 11:00:00-12:50:00 / 11:00:00-12:50:00 | F607 / F607 |
| GED0085 | MATHEMATICAL ANALYSIS | MS03 | 3 | S / W | 09:00:00-10:50:00 / 09:00:00-10:50:00 | F712 / F712 |
| GED0081 | COLLEGE PHYSICS 1 LECTURE | TN22 | 2 | F | 13:00:00-15:40:00 | ONLINE |
| GED0081L | COLLEGE PHYSICS 1 LABORATORY | TN22 | 1 | TH | 13:00:00-15:50:00 | F1003 |
| TOTAL UNITS 21 | | | | | | |

| | |
|---|---|
| Tuition Fee (21) | 44,268.00 |
| Miscellaneous Fee | 12,305.00 |
| ID Validation Fee | 80.00 |
| ITE Computer Laboratory Fee (CCS0021) | 4,008.00 |
| ITE Computer Laboratory Fee (CS0003) | 4,008.00 |
| ITE Computer Laboratory Fee (CS0070) | 4,008.00 |
| Science Lab Fee (GED0081) | 4,104.00 |
| TOTAL ASSESSMENT | 72,781.00 |
| Down Payment (Upon Enrollment) 40% | 29,112.40 |
| Midterm (Oct 05 - Oct 07 2026) 30% | 21,834.30 |
| Final (Nov 26 - Dec 03 2026) 30% | 21,834.30 |

**SOURCE CODE:**

```
; The .data section is where we store fixed text (like questions and
labels)
; and numbers that do not change.
; Note: "10" means a new line (Enter key), and "0" marks the end of
the text.
section .data
    ; --- Menu Text & File Handling Messages ---
    menu_prompt db 10
        db "======================================", 10
        db "                  COR                 ", 10
        db "======================================", 10
        db "1. Input", 10
        db "2. Output", 10
        db "3. Write", 10
        db "4. Read", 10
        db "5. Exit", 10
        db "Enter your choice: ", 0

    filename            db "cor_output.txt", 0
    p_read_file         db 10, "Enter filename to read: ", 0
    msg_write_ok        db 10, "[Details successfully written to
cor_output.txt]", 10, 0
    msg_read_hdr        db 10, "=== FILE CONTENT ===", 10, 0
    msg_file_err        db 10, "[Error: Could not open or find the
```

```
specified file.]", 10, 0
    msg_invalid         db 10, "[Invalid choice! Please select
1-5.]", 10, 0

    ; File descriptor for printing (1 = standard output / screen by
default)
    out_fd              dd 1

    ; --- Questions for Student Details (Top Part) ---
    hdr_input_top       db 10, "=== ENTER STUDENT INFORMATION ===",
10, 0
    p_status            db "Enrollment Status : ", 0
    p_student_no        db "Student #         : ", 0
    p_name              db "Name              : ", 0
    p_phone             db "Phone             : ", 0
    p_zip               db "Zip Code          : ", 0
    p_class             db "Classification    : ", 0
    p_college           db "College           : ", 0
    p_program           db "Program           : ", 0
    p_year              db "Year Level        : ", 0
    p_term_sy           db "Term & SY         : ", 0
    p_address           db "Address           : ", 0

    ; --- Questions for School Fees (Bottom Part) ---
    hdr_input_fees      db 10, "=== ENTER ASSESSMENT FEES (Tuition
fixed at 44,268.00) ===", 10, 0
    p_misc              db "Miscellaneous Fee                    :
", 0
    p_id_val            db "ID Validation Fee                    :
", 0
    p_lab_ccs0021       db "ITE Computer Lab Fee (CCS0021)       :
", 0
    p_lab_cs0003        db "ITE Computer Lab Fee (CS0003)        :
", 0
    p_lab_cs0070        db "ITE Computer Lab Fee (CS0070)        :
", 0
    p_lab_ged0081       db "Science Lab Fee (GED0081)            :
", 0

    ; --- Labels for Printing the Top Part ---
```

```
    lbl_status          db "Enrollment Status : ", 0
    lbl_student_no      db "Student # : ", 0
    lbl_college         db "College : ", 0
    lbl_program         db "   Program : ", 0
    lbl_name            db "Name : ", 0
    lbl_year            db "Year Level : ", 0
    lbl_space3          db "   ", 0
    lbl_phone           db "Phone : ", 0
    lbl_zip             db "   Zip Code : ", 0
    lbl_address         db "Address : ", 0
    lbl_class           db "Classification : ", 0
    newline             db 10, 0

    ; --- Fixed Schedule Table (Middle Part - Just prints directly)
---
    schedule_table db 10
        db
"+-------------------------------------------------------------------
-------------------------------------------------------------------+",
10
        db "| Courses   Title
Section   Units   Days    Time
Room           |", 10
        db
"+-------------------------------------------------------------------
-------------------------------------------------------------------+",
10
        db "| CCS0021   INFORMATION MANAGEMENT (LEC)            TN24
2     T       16:00:00-18:40:00                     ONLINE
|", 10
        db "| CCS0021L  INFORMATION MANAGEMENT (LAB)            TN24
1     F       16:00:00-18:50:00                     ONLINE
|", 10
        db "| CS0001    DISCRETE STRUCTURES 1                   TN24
3     S / W   15:00:00-16:50:00 / 15:00:00-16:50:00   F710 / F710
|", 10
        db "| CS0003    COMPUTER SYSTEMS & ARCHITECTURE         TN24
2     F       10:00:00-12:40:00                     ONLINE
|", 10
        db "| CS0003L   COMPUTER SYSTEMS & ARCHITECTURE (LAB)   TN24
```

```
1     TH      10:00:00-12:50:00                     E610
|", 10
        db "| CS0070    OBJECT ORIENTED PROGRAMMING            TN24
2     F       07:00:00-09:40:00                     ONLINE
|", 10
        db "| CS0070L   OBJECT ORIENTED PROGRAMMING LAB        TN24
1     TH      07:00:00-09:50:00                     E609
|", 10
        db "| GED0021   SPECIALIZED ENGLISH PROGRAM 2          TN24
3     S / W   11:00:00-12:50:00 / 11:00:00-12:50:00   F607 / F607
|", 10
        db "| GED0085   MATHEMATICAL ANALYSIS                  MS03
3     S / W   09:00:00-10:50:00 / 09:00:00-10:50:00   F712 / F712
|", 10
        db "| GED0081   COLLEGE PHYSICS 1 LECTURE              TN22
2     F       13:00:00-15:40:00                     ONLINE
|", 10
        db "| GED0081L  COLLEGE PHYSICS 1 LABORATORY           TN22
1     TH      13:00:00-15:50:00                     F1003
|", 10
        db "|
|", 10
        db "| TOTAL UNITS 21
|", 10
        db
"+-------------------------------------------------------------------
-------------------------------------------------------------------+",
10, 0

    ; --- Labels for Printing the Fees & Installments (Bottom Part)
---
    box_border          db
"+-------------------------------------------------------------------
-------+", 10, 0
    box_empty           db "|", 10, 0
    out_tuition         db "| Tuition Fee (21)
", 0
    out_misc            db "| Miscellaneous Fee
", 0
    out_id_val          db "| ID Validation Fee
```

```
", 0
    out_lab_ccs0021     db "| ITE Computer Laboratory Fee (CCS0021)
", 0
    out_lab_cs0003      db "| ITE Computer Laboratory Fee (CS0003)
", 0
    out_lab_cs0070      db "| ITE Computer Laboratory Fee (CS0070)
", 0
    out_lab_ged0081     db "| Science Lab Fee (GED0081)
", 0
    out_total           db "| TOTAL ASSESSMENT
", 0
    out_down            db "| Down Payment (Upon Enrollment) 40%
", 0
    out_midterm         db "| Midterm (Oct 05 - Oct 07 2026) 30%
", 0
    out_final           db "| Final (Nov 26 - Dec 03 2026) 30%
", 0

    ; Fixed Tuition Fee: 44,268.00 (stored in centavos as 4426800)
    tuition_fee         dd 4426800

; The .bss section reserves empty spaces (variables) where we will
save
; the user's typed answers and the calculated math results.
section .bss
    ; --- Menu & File Buffers ---
    menu_choice         resb 16
    read_filename       resb 128
    in_fd               resd 1
    file_buf            resb 4096

    ; --- Empty spaces for Student Info answers ---
    in_status           resb 64
    in_student_no       resb 64
    in_name             resb 128
    in_phone            resb 64
    in_zip              resb 32
    in_class            resb 64
    in_college          resb 64
    in_program          resb 64
```

```
    in_year             resb 32
    in_term_sy          resb 64
    in_address          resb 128

    ; --- Temporary spaces for reading and converting numbers ---
    fee_input_buf       resb 64
    num_str_buf         resb 32

    ; --- Variables to store money amounts (in centavos) ---
    misc_fee            resd 1
    id_val_fee          resd 1
    lab_ccs0021         resd 1
    lab_cs0003          resd 1
    lab_cs0070          resd 1
    lab_ged0081         resd 1
    total_assessment    resd 1
    down_payment        resd 1
    midterm_pay         resd 1
    final_pay           resd 1


; Macro 1: Prints a text message to the active output descriptor
(screen or file)
%macro PRINT_ZSTR 1
    mov esi, %1             ; Put the text address into ESI
    call print_zstr         ; Run the printing procedure
%endmacro

; Macro 2: Asks a question and saves what the user types on the
keyboard
; %1 = question text, %2 = where to save the answer, %3 = max
characters allowed
%macro PROMPT_TEXT 3
    PRINT_ZSTR %1           ; Print the question
    mov eax, 3              ; 3 tells Linux we want to read from the
keyboard
    mov ebx, 0              ; 0 means standard input (keyboard)
    mov ecx, %2             ; Where to store the typed letters
    mov edx, %3             ; Maximum number of letters to read
    int 0x80                ; Tell Linux to read the keyboard now
```

```
    mov esi, %2
    call strip_newline      ; Remove the "Enter" key at the end of
the typed text
%endmacro

; Macro 3: Asks for a fee, converts the typed text into a number, and
saves it
; %1 = question text, %2 = number variable to save it in
%macro PROMPT_FEE 2
    PROMPT_TEXT %1, fee_input_buf, 64
    mov esi, fee_input_buf
    call parse_currency     ; Convert typed text (like "80.00") into
a number (8000)
    mov [%2], eax           ; Save the number into the variable
%endmacro

; Macro 4: Prints one row of the fee table (Label + Money Amount +
New Line)
%macro PRINT_FEE_ROW 2
    PRINT_ZSTR %1           ; Print the fee name
    mov eax, [%2]           ; Load the money amount
    call print_currency     ; Print the number with commas and a
decimal point
    PRINT_ZSTR newline      ; Move to the next line
%endmacro

section .text
    global _start

_start:
.menu_loop:
    mov dword [out_fd], 1   ; Ensure menu prints to standard output
(screen)
    PROMPT_TEXT menu_prompt, menu_choice, 16

    mov al, [menu_choice]   ; Check the first character typed by the
user
    cmp al, '1'
    je .opt_input
    cmp al, '2'
```

```
    je .opt_output
    cmp al, '3'
    je .opt_write
    cmp al, '4'
    je .opt_read
    cmp al, '5'
    je .opt_exit

    ; If input is not 1-5, show error and loop back
    PRINT_ZSTR msg_invalid
    jmp .menu_loop

.opt_input:
    call get_user_inputs
    call calculate_assessment
    jmp .menu_loop

.opt_output:
    call display_full_info
    jmp .menu_loop

.opt_write:
    call write_to_file
    jmp .menu_loop

.opt_read:
    call read_from_file
    jmp .menu_loop

.opt_exit:
    mov eax, 1              ; 1 tells Linux to exit the program
    xor ebx, ebx            ; 0 means no errors happened
    int 0x80

display_full_info:
    call print_top_section
    call print_schedule_section
    call print_assessment_section
    ret
```

```
write_to_file:
    ; Create/truncate the file using sys_creat (syscall 8)
    mov eax, 8
    mov ebx, filename
    mov ecx, 0644o          ; File permissions: rw-r--r--
    int 0x80

    test eax, eax           ; Check if file descriptor is valid (>=
0)
    js .write_err           ; Jump if negative (error)

    mov [out_fd], eax       ; Redirect all PRINT_ZSTR calls to this
file
    call display_full_info  ; Print all 3 sections into the file

    ; Close the file using sys_close (syscall 6)
    mov eax, 6
    mov ebx, [out_fd]
    int 0x80

    mov dword [out_fd], 1   ; Restore output back to the screen
(stdout)
    PRINT_ZSTR msg_write_ok
    ret

.write_err:
    mov dword [out_fd], 1
    PRINT_ZSTR msg_file_err
    ret

read_from_file:
    ; Ask user which file in the current directory they want to read
    PROMPT_TEXT p_read_file, read_filename, 128

    ; Open the user-specified file in read-only mode using sys_open
(syscall 5)
    mov eax, 5
    mov ebx, read_filename
    mov ecx, 0              ; 0 = O_RDONLY
    int 0x80
```

```
    test eax, eax           ; Check if file opened successfully (>=
0)
    js .read_err            ; Jump if negative (file not found /
error)

    mov [in_fd], eax        ; Save file descriptor
    PRINT_ZSTR msg_read_hdr

.read_loop:
    ; Read chunk from file using sys_read (syscall 3)
    mov eax, 3
    mov ebx, [in_fd]
    mov ecx, file_buf
    mov edx, 4096
    int 0x80

    test eax, eax           ; Check number of bytes read (0 = EOF, <0
= error)
    jle .close_file

    ; Write the bytes read directly to the screen using sys_write
(syscall 4)
    mov edx, eax            ; EDX = exact number of bytes read
    mov eax, 4
    mov ebx, 1              ; 1 = stdout
    mov ecx, file_buf
    int 0x80
    jmp .read_loop

.close_file:
    ; Close the file using sys_close (syscall 6)
    mov eax, 6
    mov ebx, [in_fd]
    int 0x80
    ret

.read_err:
    PRINT_ZSTR msg_file_err
    ret
```

```
get_user_inputs:
    PRINT_ZSTR hdr_input_top
    PROMPT_TEXT p_status,     in_status,     64
    PROMPT_TEXT p_student_no, in_student_no, 64
    PROMPT_TEXT p_name,       in_name,       128
    PROMPT_TEXT p_phone,      in_phone,      64
    PROMPT_TEXT p_zip,        in_zip,        32
    PROMPT_TEXT p_class,      in_class,      64
    PROMPT_TEXT p_college,    in_college,    64
    PROMPT_TEXT p_program,    in_program,    64
    PROMPT_TEXT p_year,       in_year,       32
    PROMPT_TEXT p_term_sy,    in_term_sy,    64
    PROMPT_TEXT p_address,    in_address,    128

    PRINT_ZSTR hdr_input_fees
    PROMPT_FEE p_misc,        misc_fee
    PROMPT_FEE p_id_val,      id_val_fee
    PROMPT_FEE p_lab_ccs0021, lab_ccs0021
    PROMPT_FEE p_lab_cs0003,  lab_cs0003
    PROMPT_FEE p_lab_cs0070,  lab_cs0070
    PROMPT_FEE p_lab_ged0081, lab_ged0081
    ret

calculate_assessment:
    mov eax, [tuition_fee]
    add eax, [misc_fee]
    add eax, [id_val_fee]
    add eax, [lab_ccs0021]
    add eax, [lab_cs0003]
    add eax, [lab_cs0070]
    add eax, [lab_ged0081]
    mov [total_assessment], eax

    ; Calculate Down Payment (40%): (Total * 40) / 100
    mov eax, [total_assessment]
    mov ebx, 40
    mul ebx
    mov ebx, 100
    div ebx
```

```
    mov [down_payment], eax

    ; Calculate Midterm Payment (30%): (Total * 30) / 100
    mov eax, [total_assessment]
    mov ebx, 30
    mul ebx
    mov ebx, 100
    div ebx
    mov [midterm_pay], eax

    ; Calculate Final Payment (30%): (Total * 30) / 100
    mov eax, [total_assessment]
    mov ebx, 30
    mul ebx
    mov ebx, 100
    div ebx
    mov [final_pay], eax
    ret

print_top_section:
    PRINT_ZSTR newline

    ; Line 1: Enrollment Status
    PRINT_ZSTR lbl_status
    PRINT_ZSTR in_status
    PRINT_ZSTR newline

    ; Line 2: Student # on the left, College & Program on the right
    PRINT_ZSTR lbl_student_no
    PRINT_ZSTR in_student_no
    mov esi, in_student_no
    call str_len
    add eax, 12
    call pad_to_col_56
    PRINT_ZSTR lbl_college
    PRINT_ZSTR in_college
    PRINT_ZSTR lbl_program
    PRINT_ZSTR in_program
    PRINT_ZSTR newline
```

```
    ; Line 3: Name on the left, Year Level & Term on the right
    PRINT_ZSTR lbl_name
    PRINT_ZSTR in_name
    mov esi, in_name
    call str_len
    add eax, 7
    call pad_to_col_56
    PRINT_ZSTR lbl_year
    PRINT_ZSTR in_year
    PRINT_ZSTR lbl_space3
    PRINT_ZSTR in_term_sy
    PRINT_ZSTR newline

    ; Line 4: Phone & Zip Code on the left, Address on the right
    PRINT_ZSTR lbl_phone
    PRINT_ZSTR in_phone
    PRINT_ZSTR lbl_zip
    PRINT_ZSTR in_zip
    mov esi, in_phone
    call str_len
    mov ebx, eax
    mov esi, in_zip
    call str_len
    add eax, ebx
    add eax, 22
    call pad_to_col_56
    PRINT_ZSTR lbl_address
    PRINT_ZSTR in_address
    PRINT_ZSTR newline

    ; Line 5: Classification
    PRINT_ZSTR lbl_class
    PRINT_ZSTR in_class
    PRINT_ZSTR newline
    ret

print_schedule_section:
    PRINT_ZSTR schedule_table
    ret
```

```
print_assessment_section:
    PRINT_ZSTR newline
    PRINT_ZSTR box_border
    PRINT_FEE_ROW out_tuition,      tuition_fee
    PRINT_FEE_ROW out_misc,         misc_fee
    PRINT_FEE_ROW out_id_val,       id_val_fee
    PRINT_FEE_ROW out_lab_ccs0021,  lab_ccs0021
    PRINT_FEE_ROW out_lab_cs0003,   lab_cs0003
    PRINT_FEE_ROW out_lab_cs0070,   lab_cs0070
    PRINT_FEE_ROW out_lab_ged0081,  lab_ged0081
    PRINT_FEE_ROW out_total,        total_assessment
    PRINT_ZSTR box_empty
    PRINT_FEE_ROW out_down,         down_payment
    PRINT_FEE_ROW out_midterm,      midterm_pay
    PRINT_FEE_ROW out_final,        final_pay
    PRINT_ZSTR box_border
    ret

print_zstr:
    pushad
    call str_len
    test eax, eax
    jz .done
    mov edx, eax
    mov ecx, esi
    mov ebx, [out_fd]       ; Uses out_fd (1 for screen, or file
descriptor)
    mov eax, 4
    int 0x80
.done:
    popad
    ret

str_len:
    push esi
    xor eax, eax
.loop:
    cmp byte [esi], 0
    je .done
    inc esi
```

```
    inc eax
    jmp .loop
.done:
    pop esi
    ret

strip_newline:
    push esi
.loop:
    mov al, [esi]
    test al, al
    jz .done
    cmp al, 10
    je .replace
    cmp al, 13
    je .replace
    inc esi
    jmp .loop
.replace:
    mov byte [esi], 0
.done:
    pop esi
    ret

pad_to_col_56:
    pushad
    mov ecx, 56
    sub ecx, eax
    cmp ecx, 3
    jge .print_spaces
    mov ecx, 3
.print_spaces:
    mov byte [num_str_buf], ' '
    mov byte [num_str_buf + 1], 0
.loop:
    PRINT_ZSTR num_str_buf
    loop .loop
    popad
    ret
```

```
parse_currency:
    xor eax, eax
    xor ebx, ebx
    xor ecx, ecx
.int_loop:
    movzx edx, byte [esi]
    test dl, dl
    jz .finish
    inc esi
    cmp dl, ','
    je .int_loop
    cmp dl, ' '
    je .int_loop
    cmp dl, '.'
    je .frac_loop
    cmp dl, '0'
    jb .int_loop
    cmp dl, '9'
    ja .int_loop
    sub dl, '0'
    imul eax, eax, 10
    add eax, edx
    jmp .int_loop

.frac_loop:
    cmp ecx, 2
    jae .finish
    movzx edx, byte [esi]
    test dl, dl
    jz .finish
    inc esi
    cmp dl, '0'
    jb .frac_loop
    cmp dl, '9'
    ja .frac_loop
    sub dl, '0'
    imul ebx, ebx, 10
    add ebx, edx
    inc ecx
    jmp .frac_loop
```

```
.finish:
    cmp ecx, 1
    jne .combine
    imul ebx, ebx, 10
.combine:
    imul eax, eax, 100
    add eax, ebx
    ret

print_currency:
    pushad
    mov edi, num_str_buf + 31
    mov byte [edi], 0

    xor edx, edx
    mov ebx, 100
    div ebx

    push eax
    mov eax, edx
    xor edx, edx
    mov ebx, 10
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    add al, '0'
    dec edi
    mov [edi], al
    dec edi
    mov byte [edi], '.'
    pop eax

    xor ecx, ecx
    test eax, eax
    jnz .peso_loop
    dec edi
    mov byte [edi], '0'
    jmp .out
```

```
.peso_loop:
    cmp ecx, 3
    jne .extract_digit
    dec edi
    mov byte [edi], ','
    xor ecx, ecx
.extract_digit:
    xor edx, edx
    mov ebx, 10
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    inc ecx
    test eax, eax
    jnz .peso_loop

.out:
    mov esi, edi
    call print_zstr
    popad
    ret
```